Green University of Bangladesh
Department of Computer Science and Engineering (CSE)
Faculty of Sciences and Engineering
Semester: (Summer, Year:2026), B.Sc. in CSE (Day)

Lab Report NO : 2
Course Title: Artificial Intelligence Lab
Course Code: CSE 316 Section: 241-D2

Lab Experiment Name: Implementation of DFS Algorithm.

Student Details

	Name	ID
1.	Md. REAHOON ZANNAH	223002038

Lab Date : 29-06-2026
Submission Date : 06-07-2026
Course Teacher's Name : Fatema Akter

Lab Report Status

Marks: ……. **Signature:.....**
Comments:..... **Date:.....**

1. TITLE OF THE LAB REPORT EXPERIMENT

Implementation of DFS Algorithm.

2. OBJECTIVES

- To understand the basic concept of the DFS algorithm.
- To implement the DFS algorithm in Python.
- To find a path from the start node to the goal node.

3. PROCEDURE

- ❖ Open the Python IDE (Google Colab).
- ❖ Define the fixed 6 × 6 maze grid with free cells (0) and obstacles (1).
- ❖ Set the starting position as (0, 0) and the goal position as (5, 5).
- ❖ Implement the DFS algorithm using recursion and a visited matrix.
- ❖ Traverse the maze following the movement order: Down → Right → Up → Left.
- ❖ Stop the search when the goal node is reached and record the traversal path.
- ❖ Analyze the number of moves and the path taken to reach the goal.

4. IMPLEMENTATION:

a) **Robot path nevigation for 6*6 grid in DFS:**

```
grid = [
    [0, 0, 1, 0, 0, 0],
    [1, 0, 1, 0, 1, 0],
    [0, 0, 0, 0, 1, 0],
    [0, 1, 1, 0, 0, 0],
    [0, 0, 0, 1, 1, 0],
    [1, 1, 0, 0, 0, 0]
]

n = 6
vis = [[0] * n for _ in range(n)]
path = []
moves = [(1, 0, "Down"), (0, 1, "Right"), (-1, 0, "Up"), (0, -1, "Left")]

def dfs(x, y):
    if (x, y) == (n - 1, n - 1):
        return True
    vis[x][y] = 1
    for dx, dy, d in moves:
        nx, ny = x + dx, y + dy
```

```
        if (0 <= nx < n and 0 <= ny < n and grid[nx][ny] == 0 and not
vis[nx][ny]):
            path.append((d, nx, ny))
            if dfs(nx, ny):
              return True
            path.pop()
    return False

if grid[0][0] == 0 and dfs(0, 0):
    print("Goal found")
    print("Number of moves required =", len(path))
    for d, x, y in path:
        print(f"Moving {d} ({x}, {y})")
else:
    print("Goal not found")
```

5. OUTPUT

Fig-1.1: Result of (a).

6. ANALYSIS AND DISCUSSION

The Depth First Search methodology was effectively applied and run within the Python environment. By systematically exploring the grid using the designated directional sequence: Down, Right, Up, and Left, the script identified an appropriate route from origin to destination while bypassing all barriers. The resulting output aligned with predicted benchmarks, illustrating how DFS serves as a functional solution for navigating complex path-finding scenarios.

7. SUMMARY

The lab experiment provided practical experience in implementing the Depth-First Search (DFS) algorithm using Python. The program successfully navigated a maze, found the goal node, and displayed the traversal path and total number of moves. The experiment improved understanding of DFS recursion, and logical problem-solving techniques.

8. GITHUB REPOSITORY:

Link: https://github.com/pro382r/GUB-Academic/tree/main/AI-lab/lr2